SWARM: Scientific Workflow Applications on Resilient Metasystem

Ewa Deelman*, Prasanna Balaprakash†, Franck Cappello§, Anirban Mandal‡, Krishnan Raghavan§,
Hongwei Jin§, Komal Thareja‡, Jean Luca Bez ¶, Frédéric Sutter †, Devesh Tiwari // , Hamza Safri*
*University of Southern California, †Oak Ridge National Laboratory, ‡Renaissance Computing Institute, §Argonne National Laboratory, ¶Lawrence Berkeley National Laboratory, // Northeastern University

Supported by DOE Award #DE-SC0024387

Introduction

Current scientific workflow management approaches are often **centralized** and **static**, limiting **scalability**, **adaptability**, and **resilience** across heterogeneous computing environments. The **SWARM** project introduces a **decentralized, multi-agent framework** that integrates **distributed scheduling**, **resilient coordination**, **reinforcement learning (RL)**, **AI-driven decision-making**, and **chaos engineering**. SWARM develops novel strategies for **fault tolerance**, **adaptive orchestration**, and **autonomous resource management** to enable **robust, scalable, and resilient** scientific workflow execution across future **distributed cyberinfrastructure**.

Distributed Multi-Agent Scheduling

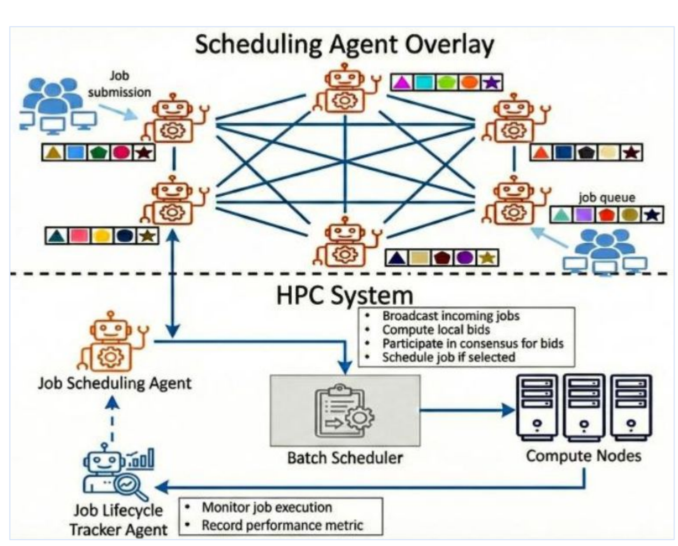
Develop a **decentralized multi-agent scheduling framework** that enables autonomous coordination across heterogeneous HPC facilities, eliminating centralized bottlenecks while improving scalability, resilience, and workflow performance.

- **Distributed architecture:** Autonomous site-level agents collaboratively schedule jobs through competitive bidding.
- **Intelligent scheduling:** Evaluate heuristic-, embedding-, and LLM-based bidding strategies for adaptive resource selection.
- **Federated execution:** Preserve local scheduling policies while optimizing resource utilization across multiple HPC systems.

Validated Decentralization
Matches the scheduling quality of centralized approaches

3× Lower Turnaround Time
Reduces median workflow turnaround through federated job placement

Best Overall Strategy
Embedding-based bidding achieves the best balance of quality and overhead

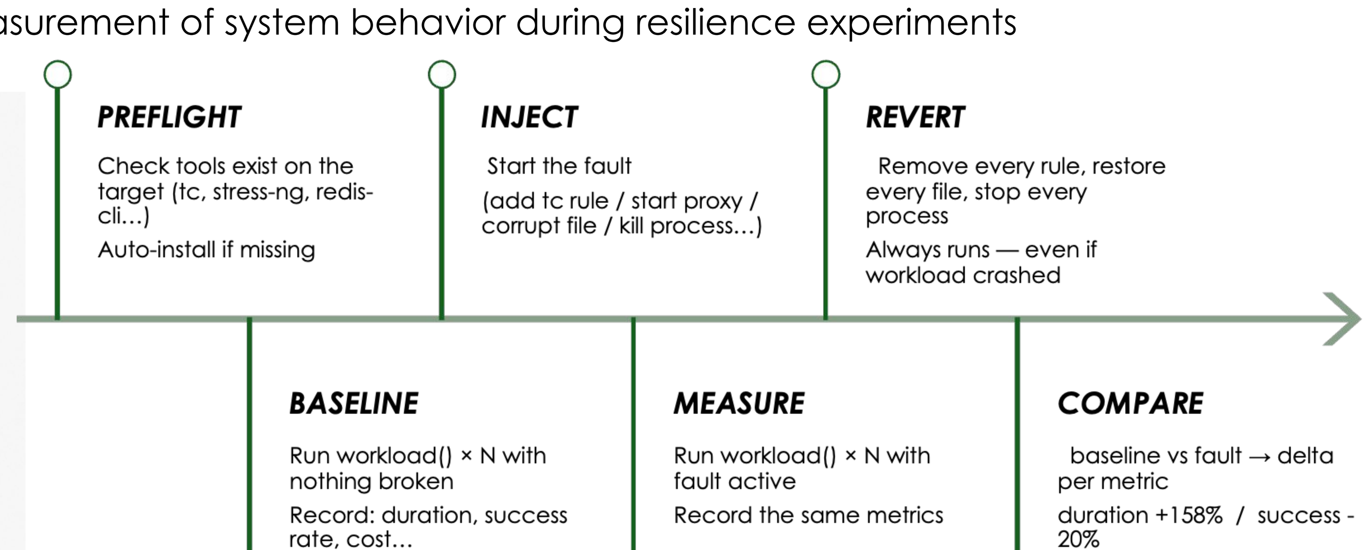


Future Direction : LLM-based bidding shows promise but remains limited by inference latency

Chaos Jungle: Automated Chaos Engineering

Chaos Jungle is a Python framework for **fault injection** and **chaos engineering** that supports controlled resilience experiments across **infrastructure, application, and AI/LLM layers**. It provides a structured experiment lifecycle for fault injection, measurement, recovery, and comparison, enabling reproducible evaluation of system behavior under faults.

- Structured experiment lifecycle: **Baseline → Inject → Measure → Recover → Compare**
- Fault injection across **infrastructure, applications, and AI/LLM systems**
- Supports **50+ fault types**, including network, storage, processes, APIs, and AI agents
- Quantitative measurement of system behavior during resilience experiments



Future Direction : Evaluating Chaos Jungle on SWARM use cases and extending support for additional fault injection scenarios

Goals

- Design a novel, performant, and resilient workflow execution platform for modern large-scale distributed computing facilities.
- Improve the resilience and performance of workflows executing across multiple HPC facilities and distributed resources.
- Empower researchers to achieve greater efficiency in their scientific pursuits

Hybrid PBFT–Snow Consensus

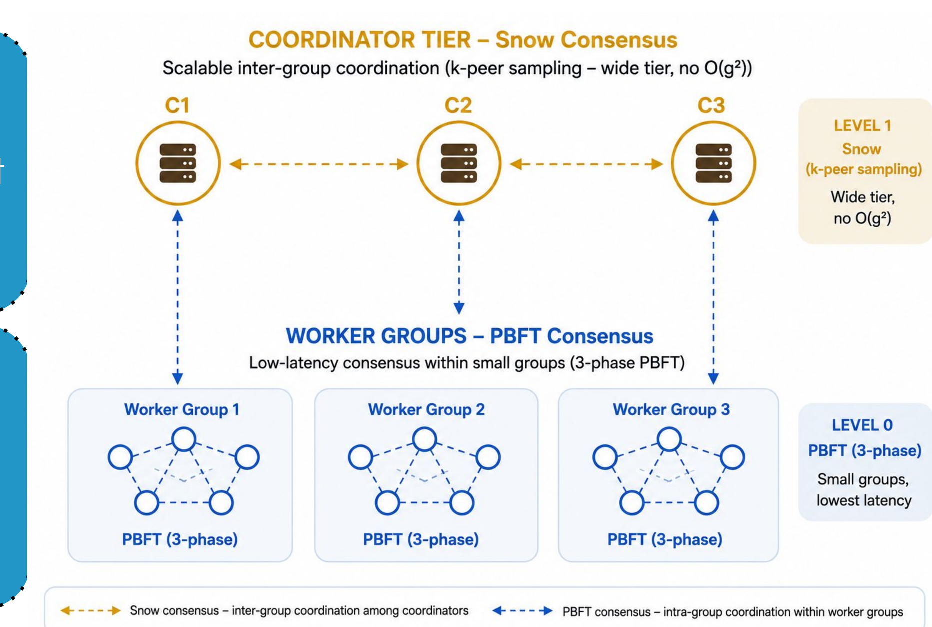
Extending the hierarchical PBFT consensus architecture [1], a hybrid consensus framework integrates **Snow gossip-based consensus** for scalable inter-group coordination while preserving **PBFT** for low-latency consensus within worker groups. Combined with **SWIM-based failure detection** and **gossip state dissemination**, the framework improves the scalability and resilience of decentralized workflow coordination across federated HPC systems.

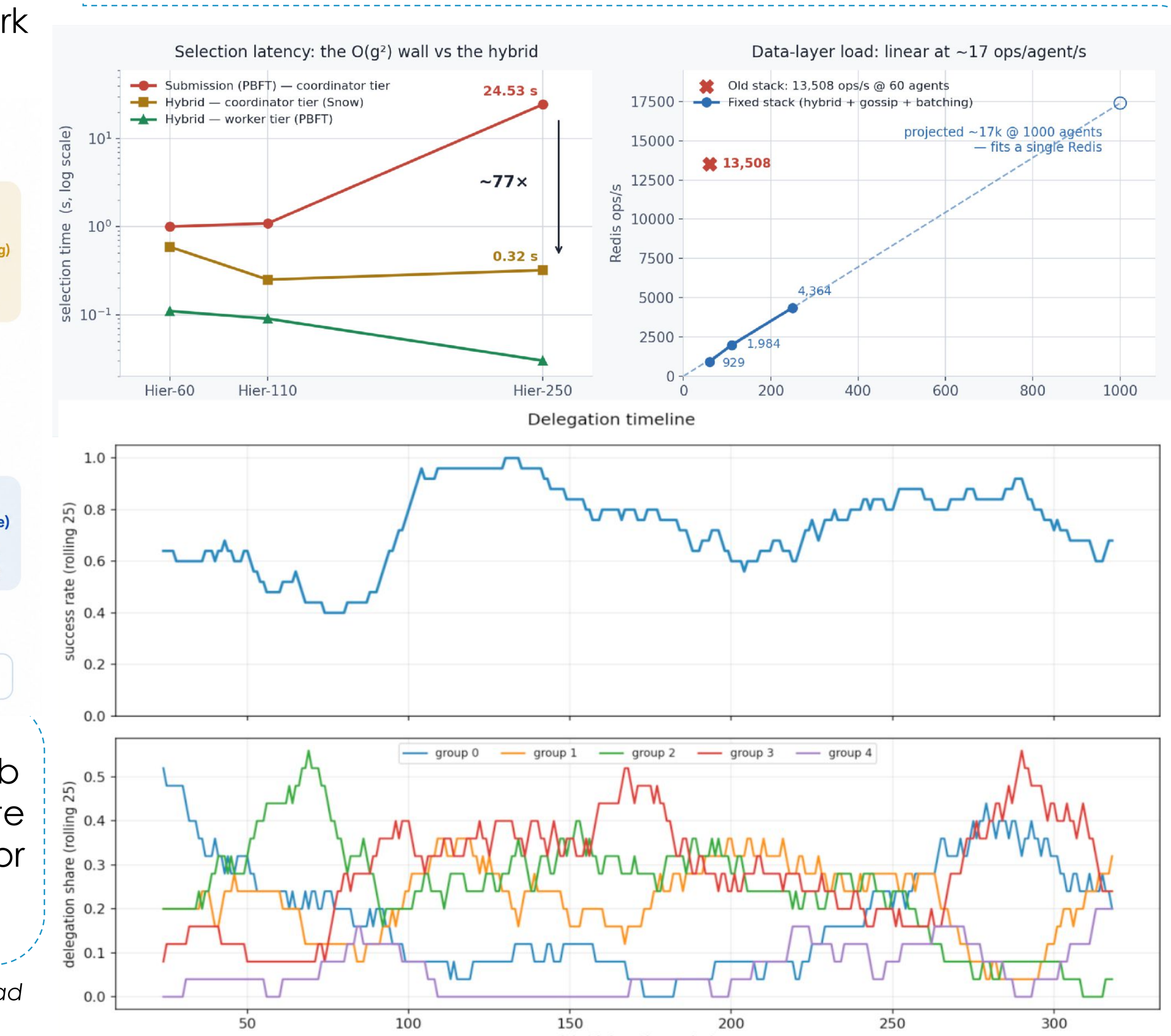
77× Faster Consensus
Reduced coordinator consensus latency from **24.53 s** to **0.32 s**

Scalable Coordination
Eliminated the coordinator bottleneck, enabling efficient consensus across large federated HPC systems

Efficient Communication
Reduced communication overhead through decentralized failure detection and gossip-based coordination

Resilient Execution
Achieved **100%** workflow completion under worker failures through decentralized recovery





Future Direction: LinUCB contextual bandits are being investigated to enable adaptive, workload-aware job delegation by learning optimal worker selection from job and system characteristics. Preliminary results demonstrate +11.5 percentage point higher job success and 69% routing accuracy, highlighting the potential of online learning for decentralized workflow scheduling.

[1] Thareja, K., Raghavan, K., Safri, H., Mandal, A., & Deelman, E. (2026, March 19). SWARM+: Scalable and Resilient Multi-Agent consensus for decentralized Data-Aware workload management. arXiv.org. https://arxiv.org/abs/2603.19431

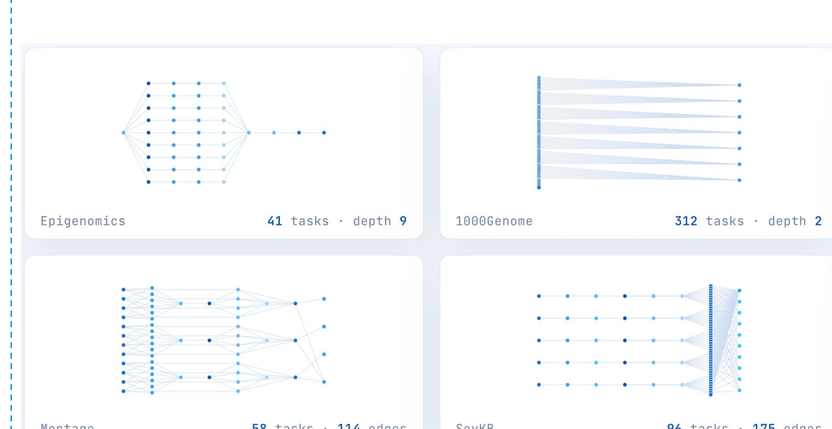
Resilient RL for Scientific Workflows

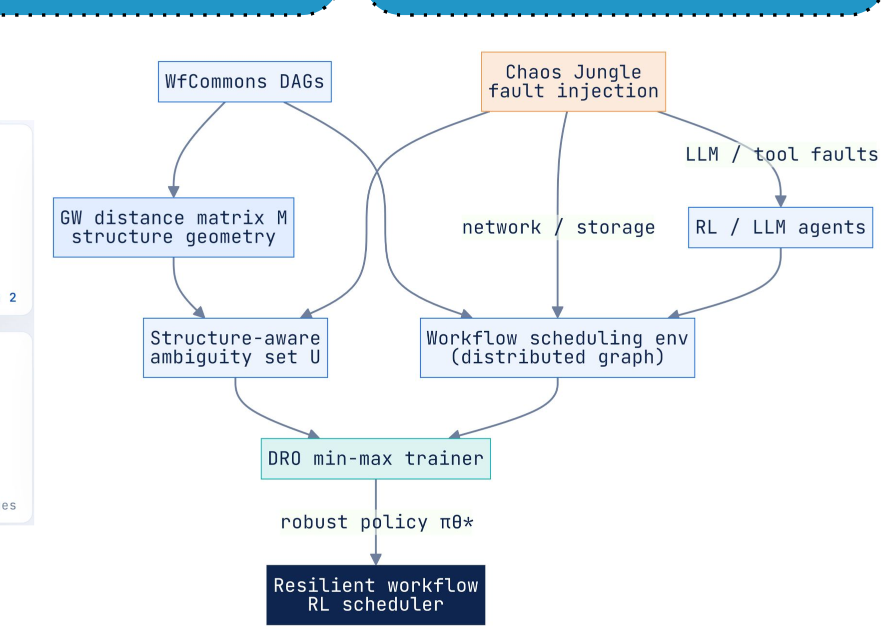
Distributionally robust reinforcement learning (RL) is being developed for adaptive workflow scheduling under uncertainty. The framework integrates **Chaos Jungle** for realistic fault injection, **distributionally robust optimization (DRO)** for worst-case policy optimization, and **Gromov–Wasserstein (GW)** geometry to enable policy transfer across **heterogeneous workflow structures** with varying depth, width, and connectivity.

~7× Fewer LLM Calls
Achieves **~0.6 LLM calls/job** compared with **4.0** for an always-LLM policy

Robust Under Network Faults
Maintains comparable makespan under **0–20% WAN packet loss** on the FABRIC testbed

Comparable Makespan
Matches always-LLM scheduling performance while reducing LLM usage





Future Direction : integrating Chaos Jungle fault campaigns into robust RL training and evaluating policy transfer across heterogeneous scientific workflow DAGs

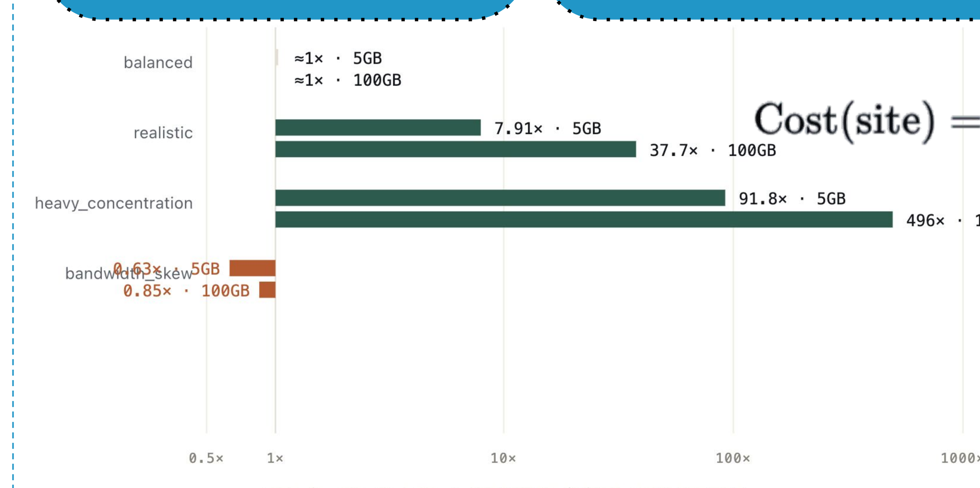
Data-Aware Scheduling

Current workflow schedulers typically optimize resource availability while ignoring data location and network transfer costs. A **data-aware scheduler** extends SwarmAgents by incorporating **data locality**, **network bandwidth**, and **compute heterogeneity** into distributed scheduling decisions, reducing unnecessary data movement across federated HPC resources.

496× Faster Scheduling
Up to **496×** speedup over the default scheduler in locality-intensive workloads

37.7× Speedup
Achieved **37.7×** improvement on representative 100 GB workloads

Bandwidth-Aware Scheduling
Identified a bandwidth-skew scenario (**0.85×**) that motivates ongoing improvements to the scheduling cost model



$$\mathrm{Cost(site)} = \underbrace{\sum \frac{\text{file size}}{\text{bandwidth(source} \to \text{site)}}}_{\textbf{Transfer}} + \alpha \underbrace{\left(\text{runtime} \times \frac{\mathrm{REF_{cores}}}{\mathrm{target_{cores}}}\right)}_{\textbf{Execution}} + \beta \underbrace{\text{(site utilization)}}_{\textbf{Capacity}}$$

Future Direction : Refine bandwidth-aware cost modeling and evaluate the scheduler across larger federated deployments and more diverse data-placement scenarios